Lab 6: Code and Branch Coverage Analysis

Name: Arundhathi K
SRN: PES1UG24CS082
Section: 5-B
Application: Pizza Order

1. Objective

To measure code coverage and branch coverage for the assigned pizza-order program and write additional tests to achieve at least 90% in both. Separate Python files are maintained for the two test suites.

2. Program description

The program contains three functions. pizza_price calculates the price using the pizza size, number of toppings and cheese-burst option. order_discount calculates the discount based on the order total, day and student status. delivery_status classifies delivery using the time taken and distance. The application code was not modified.

3. Coverage results

Test suite	Tests passed	Code coverage	Branch coverage
Original starter tests	2	14/34 (41.18%)	6/24 (25.00%)
Code coverage tests	14	34/34 (100%)	Not measured
Branch coverage tests	31	34/34 (100%)	24/24 (100%)

The first row shows the initial coverage from the supplied tests. The next two rows show the final results after adding test cases. Coverage is measured only for pizza_order.py. Code coverage measures executed statements, while branch coverage measures the outcomes taken at decisions.

4. Test cases added

Pizza price: valid pizza sizes, invalid size, negative and zero toppings, three and four toppings, additional toppings, cheese burst, and rejection of cheese burst for a small pizza.

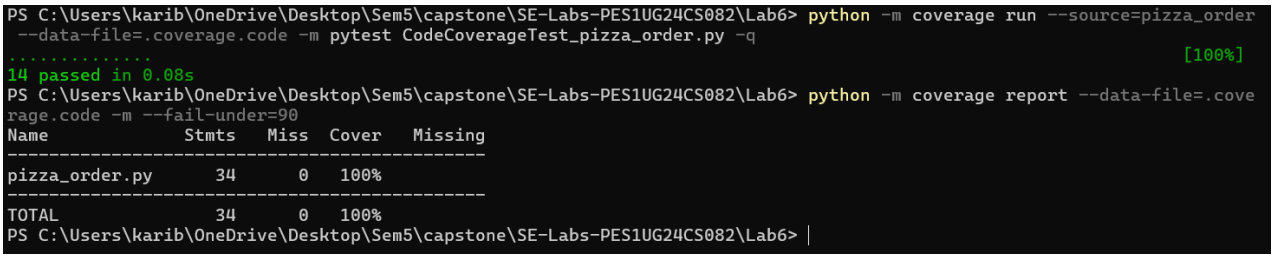
Discount: totals below and at 300, Tuesday discounts, both weekend days, regular weekdays, student and non-student orders, and integer-division boundaries.

Delivery: the 30/31-minute and 45/46-minute boundaries, a distance of exactly 10 km, and distances above 10 km.

In a branch-enabled coverage.py report, the Cover column combines statements and branches. The separate branch check confirms the branch-only result: 24 out of 24 outcomes, or 100%.

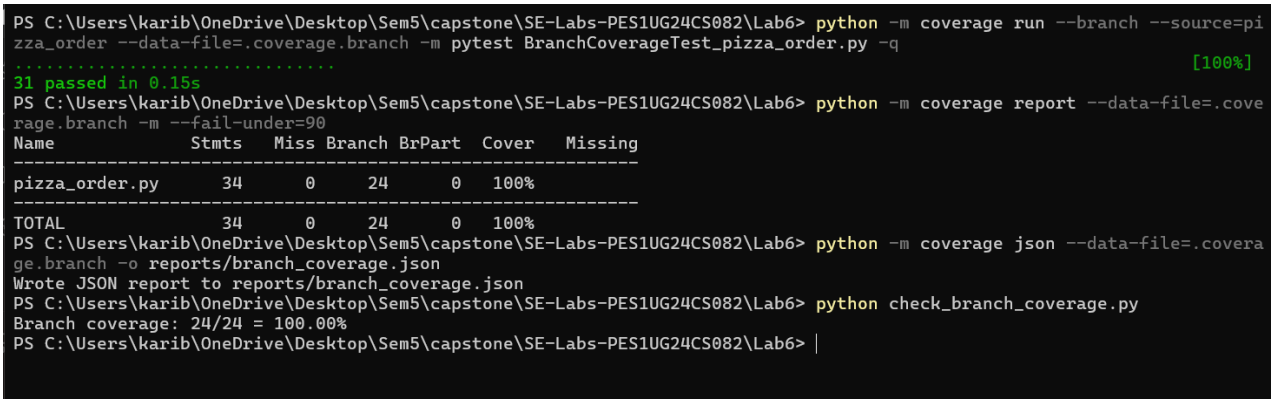
5. Code coverage - terminal screenshot

Test file: CodeCoverageTest_pizza_order.py
Result: 14 tests passed; 34 statements executed, 0 missed; code coverage = 100%.



6. Branch coverage - terminal screenshot

Test file: BranchCoverageTest_pizza_order.py
Result: 31 tests passed; 24 branch outcomes covered, 0 partial branches; branch coverage = 100%.



7. Conclusion

The additional tests increased statement coverage from 41.18% to 100% and branch coverage from 25% to 100%. Both results exceed the required minimum of 90%. Separate code coverage and branch coverage test files, terminal screenshots and this report are included in the Lab6 folder.

8. Repository and commits

Repository: github.com/ArundhathiK29/SE-Labs-PES1UG24CS082/tree/main/Lab6

Code coverage commit: 9331987
Branch coverage commit: f52029f

Assigned project: github.com/SETAPESU26/L6_L7_24_pizza_order

Appendix A: Code coverage test file

CodeCoverageTest_pizza_order.py

```
"""Statement coverage: execute every executable line in the assigned module."""
import pytest
from pizza_order import pizza_price, order_discount, delivery_status


@pytest.mark.parametrize("size,toppings,cheese,expected", [
    ("medium", 0, False, 350),
    ("small", 3, False, 320),
    ("large", 4, True, 750),
])
def test_pizza_prices(size, toppings, cheese, expected):
    assert pizza_price(size, toppings, cheese) == expected


@pytest.mark.parametrize("size,toppings,cheese,message", [
    ("extra-large", 0, False, "unknown size"),
    ("medium", -1, False, "invalid toppings"),
    ("small", 0, True, "cheese burst not available for small"),
])
def test_invalid_pizza_orders(size, toppings, cheese, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        pizza_price(size, toppings, cheese)


@pytest.mark.parametrize("total,day,student,expected", [
    (299, "tuesday", True, 0),
    (600, "tuesday", True, 350),
    (600, "saturday", False, 0),
    (600, "monday", False, 60),
])
def test_discount_statements(total, day, student, expected):
    assert order_discount(total, day, student) == expected


@pytest.mark.parametrize("minutes,distance,expected", [
    (20, 11, "not deliverable"),
    (30, 10, "on time"),
    (45, 5, "late"),
    (46, 5, "free pizza"),
])
def test_delivery_returns(minutes, distance, expected):
    assert delivery_status(minutes, distance) == expected
```

Appendix B: Branch coverage test file

BranchCoverageTest_pizza_order.py

```
"""Decision coverage with threshold pairs and independent expected values."""
import pytest
from pizza_order import pizza_price, order_discount, delivery_status


@pytest.mark.parametrize("size,toppings,cheese,expected", [
    ("small", 0, False, 200),
    ("medium", 0, False, 350),
    ("large", 0, False, 500),
    ("medium", 3, False, 470),  # Last full-price topping.
    ("medium", 4, False, 500),  # First reduced-price topping.
    ("large", 6, False, 710),
    ("medium", 0, True, 450),
    ("large", 4, True, 750),
])
def test_valid_price_branches(size, toppings, cheese, expected):
    assert pizza_price(size, toppings, cheese) == expected


@pytest.mark.parametrize("size,toppings,cheese,message", [
    ("invalid", 0, False, "unknown size"),
    ("medium", -1, False, "invalid toppings"),
    ("small", 0, True, "cheese burst not available for small"),
])
def test_validation_branches(size, toppings, cheese, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        pizza_price(size, toppings, cheese)


@pytest.mark.parametrize("total,day,student,expected", [
    (0, "monday", False, 0),
    (299, "tuesday", True, 0),
    (300, "tuesday", False, 150),
    (300, "tuesday", True, 200),
    (301, "tuesday", False, 150),  # Integer division floors.
    (300, "saturday", False, 0),
    (300, "saturday", True, 50),
    (300, "sunday", False, 0),
    (300, "sunday", True, 50),
    (300, "monday", False, 30),
    (300, "monday", True, 80),
    (309, "friday", False, 30),
    (310, "friday", False, 31),
])
def test_discount_decisions(total, day, student, expected):
    assert order_discount(total, day, student) == expected


@pytest.mark.parametrize("minutes,distance,expected", [
    (0, 0, "on time"),
    (30, 10, "on time"),
    (31, 10, "late"),
    (45, 10, "late"),
    (46, 10, "free pizza"),
    (20, 10.01, "not deliverable"),
    (46, 11, "not deliverable"),  # Distance decision takes priority.
])
def test_delivery_decisions(minutes, distance, expected):
    assert delivery_status(minutes, distance) == expected
```